How the circuit bank works

Isomorphic DC-circuit problem banks: what “isomorphic” means, why it is built for fairness to the student, and how the code produces 30 variations at one difficulty with every number computed, not guessed.

Myrla Phillippe · research-group presentation notes · October 2026

In one sentence. Take Zhongzhou Chen's isomorphic-bank method, feed it a domain where the “scenario” is a circuit topology that a computer can enumerate and verify, and let a deterministic circuit engine do everything numerical — the result is a bank where every item has a different solution path, the same difficulty (exactly three reduction steps, five or six resistors), computed wrong answers, and an answer key that regenerates byte-for-byte.

Suggested slide order (each section below is one or two slides): 1 What isomorphic means · 2 Built for the student · 3 The three pieces · 4 Walk-through of one run · 5 Where the AI is and is not · 6 Difficulty has a ceiling · 7 Results and verification · 8 Limits and next steps.

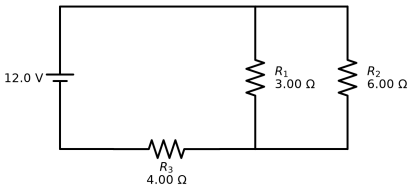
SLIDE 1

1. What “isomorphic” means here

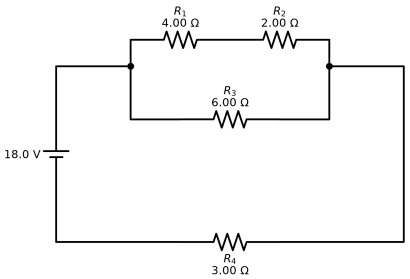
The word is Zhongzhou Chen's. His repository defines an isomorphic bank as problems that test the same learning objectives “with variations that change the solution and not only the numbers”, drawn at random so no two students get the same item. Spelled out, that is three criteria, and all three matter:

	Criterion	What it rules out
1	The variation is more than changing numbers or swapping nouns.	“Same circuit, new resistor values.” Students pattern-match the one solution path.
2	It changes the solution — at least one step is different.	Cosmetic changes: redrawing the figure, re-ordering parts, renaming $R_1 \to R_2$.
3	It tests the same learning objective at the same difficulty.	Mixing a 2-step circuit with a 5-step one, or asking for current in one item and power in another.

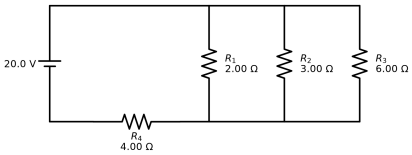
For a resistor network the natural reading is: an isomorph is a different series-parallel topology — a different sequence of reductions — asked the same way, at the same number of steps. Three real items from the 10-item bank:



`3 || 6 + 4`
parallel pair, then series



`(4 + 2) || 6 + 3`
series pair inside a parallel, then series



`2 || 3 || 6 + 4`
three-way parallel, then series

Same question (“find the total current”), same tools (series/parallel reduction + Ohm's law), but no two of them can be solved by remembering the other. That is the floor. The ceiling — same difficulty — is handled in section 6.

> Why the word matters. These banks exist for one use: each student draws one random item from the bank for the same exam slot. Below the floor the draw is pointless (everyone has the same problem). Above the ceiling the draw is unfair (one student's exam is harder). Isomorphic is exactly the band where a random draw is both meaningful and fair.

SLIDE 2

2. Built with the student in mind

Design choice	What the student experiences	How it is enforced
One difficulty per bank	Whoever you are, your drawn problem takes the same number of reduction steps (3) with the same number of resistors (5–6) as your neighbour's.	`reduction_steps: 3`, `min/max_resistors: 5/6` in the input; the critic rejects any candidate outside them. One step = one series run or parallel group collapsed into a single resistor.
Every item a different solution path	You cannot pass by memorising last year's problem; you have to reduce the network you were given.	Canonical-shape check: two circuits with the same tree shape are the same problem, whatever the numbers.
Wrong answers are real mistakes	Three distractors are the numbers you get by making one specific error each; a fifth option sits near the largest value so a lone big number is not a giveaway by size.	`distractors.py` recomputes the circuit as a mistaken student would (all-series, forgot-to-invert, branch current) and adds a decoy at 0.9 × the largest option.
Honest sizes	A distractor of 144 A is what the mistake really gives; its implausibility is itself feedback.	Deliberately not capped (a documented decision).
One drawing style	The figure never changes style between items, so the only thing that varies is the physics.	`layout: ladder` for the whole bank; style is never a variation axis.
Clean arithmetic	Every intermediate resistance is a whole or half ohm; answers have at most two decimals.	The enumerator only accepts value sets that satisfy this; resistor pool ≤ 12 Ω.
Worked solution for every item	After the exam, the feedback shows every reduction step and a per-resistor table.	Produced by the circuit engine, stored in `feedback.general`.

Example of computed options — item q1 of the 10-item bank (12 V, `3 || 6 + 4`, correct 2.00 A):

Option	Where it comes from
0.92 A	adds every resistor as if in series: 12 / (3 + 6 + 4)
2.67 A	adds the reciprocals of the parallel pair but forgets to invert: R_{12} = 1/3 + 1/6 = 0.5 “Ω”
1.33 A	reports the current through R_1 (one branch) instead of the total
2.40 A	decoy: 0.9 × the largest option, so the large value is not alone
2.00 A	correct: 3 ‖ 6 = 2 Ω, + 4 = 6 Ω, 12 V / 6 Ω

SLIDE 3

3. The three pieces

Piece	Author	What it contributes	AI inside?
Isomorphic-bank method isomorphic-bank-creation-skills	Zhongzhou Chen	The definition of isomorphic; an input scheme; a generate → critique → count loop with a target, accepted/rejected lists with reasons and a 5-pass cap (`scenario_list.py`); the bank YAML format and its folder convention (bank id, QTI zip, figures).	His critic and scenario writer are LLM agents. His docs state numbers, wording and figures are "not generated yet."
circuit-diagram-skill	Myrla Phillippe	A small language ("circuit lingo": `(4 + 2) || 6 + 3`), a parser, a series/parallel solver with worked steps, and a Schemdraw figure generator. Same text in → same figure, numbers and solution out.	None. An LLM may only translate an unreadable English sentence into lingo; it never invents circuits.
circuit-bank skill	this project	Runs Chen's loop for circuits: an enumerator proposes every shape in the difficulty band; a script critic replaces his LLM critic; the circuit engine replaces his sympy stage; computed distractors; bank YAML + PDF output.	None by default. An LLM proposer exists as an opt-in (`generator: llm`) for comparison.

Chen's repository separates the tooling (his two Cursor skills) from the banks (one folder per bank, in his YAML template). Our output follows the same template and folder convention, so a circuit bank sits beside his mechanics banks without translation.

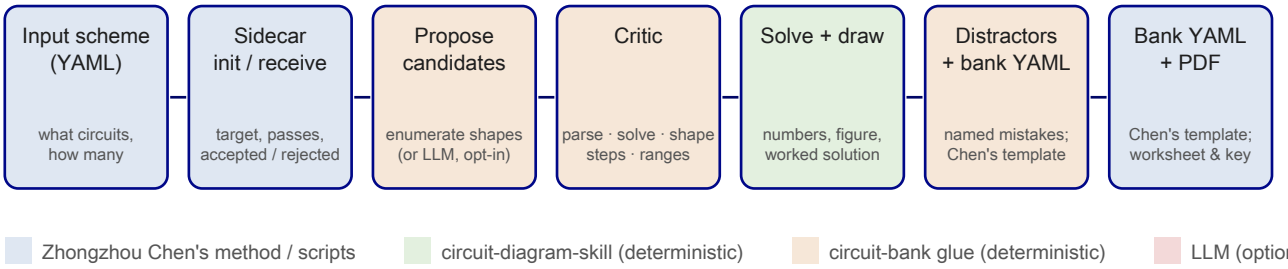


Figure: the pipeline, coloured by whose work each stage is.

SLIDE 4

4. Walk-through of one run (the 30-item bank)

4.1 The input scheme

Chen's format, plus a `bank:` block and a few machine-checkable ranges. The number of items lives here (`n_scenarios: 30`), never in a chat message — one of his rules we kept.

```
name: "Series-parallel resistor networks: total current (3 reduction steps)"
- Every scenario is a different series/parallel topology (a different reduction
sequence), not only different numbers.
- Between 5 and 6 resistors, and every circuit reduces in exactly 3 series/parallel
steps (one difficulty for the whole bank).
- Ask for the total current from the battery.
n_scenarios: 30
bank_id: PHY2-DC-SPRTC3-10292026
question_type: multiple_choice
layout: ladder
resistor_range: [1, 24]
voltage_range: [6, 36]
max_resistors: 6
min_resistors: 5
reduction_steps: 3
generator: enumerate
seed: 0
```

4.2 Chen's loop opens the sidecar

`validate_input.py` (his Agent 1 check) passes the file; `scenario_list.py init` copies the target; `receive` starts pass 1 and reports `needed: 30`.

4.3 Propose candidates — the enumerator

Every series-parallel network is a tree whose internal nodes alternate series and parallel. The enumerator generates all such trees with 5–6 resistors, keeps those with exactly 3 internal nodes (= 3 reduction steps), and skips shapes already accepted. There are 30 of them (appendix A lists all) — so a 30-item bank uses every one. For each shape it then picks resistor values from a fixed pool (1–12 Ω) and a battery voltage, using a seed, until every intermediate resistance is a multiple of 0.5 Ω, the answer has ≤ 2 decimals, and no earlier item has the same answer. Output: 30 three-line lingo strings — the same JSON an LLM proposer would return.

```
source: 9 V
circuit: (6 + 4) || (9 + 6)
ask: total current
```

4.4 The critic — a script, not an opinion

Each candidate is parsed and solved by the circuit engine; it is rejected, with a reason written into the sidecar, if any of these fail:

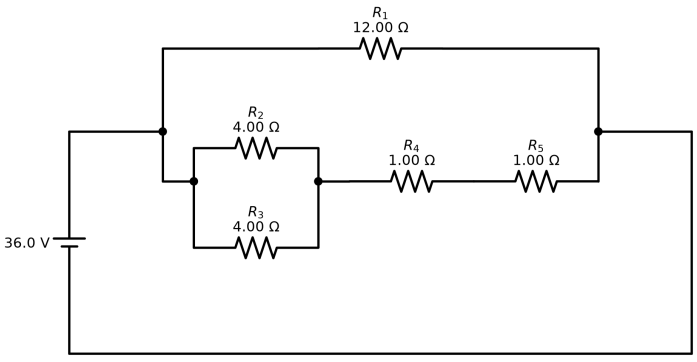
Check	Rejection reason as written to the sidecar
parses and solves	circuit-diagram-skill rejects it: The circuit line ends too early…

Check	Rejection reason as written to the sidecar
new canonical shape (+ ask)	same solution path (S(P(R,R),R) ask total_current) as an accepted scenario: 3 || 6 + 4
values in range	outside the input's ranges: R1 = 50 Ω is outside resistor_range [1, 24]
difficulty	outside the input's ranges: 5 reduction steps; this bank wants 3 (same difficulty for every item)

The canonical shape ignores the order of siblings, so `3 || 6 + 4`, `6 || 3 + 4` and `2 || 4 + 5` all collapse to `S(P(R,R),R)` — one problem, not three. With the enumerator proposing, the critic is a safety net: in this run it kept 30 of 30 on the first pass.

4.5 Numbers, figure, worked solution, distractors

For each accepted lingo the circuit engine draws the figure (ladder layout, US symbols) and writes the reduction steps, the per-part table and the answer. `distractors.py` re-walks the same tree three wrong ways and adds a decoy near the largest option. Everything is assembled into one bank YAML in Chen's template (so it can be imported wherever his banks are) and rendered to a PDF worksheet with an answer key.



Item q2 of the 30-item bank: `12 || (4 || 4 + 1 + 1)`, 36 V → 12.00 A. Distractors: 74.48 A, 9.00 A, 67.03 A, 1.64 A.

SLIDE 5

5. Where the AI is — and is not

Stage	Chen's pipeline	This pipeline
Describe the problem family	human writes the input	human writes the input
Propose variations	LLM writes prose scenarios	enumeration (all shapes in the band); LLM opt-in
Judge variations	LLM critic	script: parse, solve, shape, steps, ranges
Equations / numbers	sympy from LaTeX formulas; random numbers “not generated yet”	circuit engine, deterministic
Figures	not generated	Schemdraw from the same lingo
Wrong answers	—	computed from named mistakes
Wording	not generated	fixed template from the engine

Reproducibility. Same input + same seed → the same 30 scenarios. Same scenario sidecar → the same bank, byte for byte (verified: regenerating produced an identical SVG). The sidecar is the bank's source of truth; the PDF, the figures and the YAML are all derived from it.

When the LLM proposer is used (we ran it for the first 10-item bank), correctness is unchanged — the critic and the engine do not care who wrote the string — but two guarantees are lost: the set of shapes differs from run to run, and completeness is not assured. That mattered as soon as we fixed one difficulty (next section): finding all 30 possible shapes is a job for enumeration, not guessing.

SLIDE 6

6. Difficulty has a ceiling

Fixing one difficulty means fixing the number of reduction steps. The number of distinct series-parallel shapes is finite and small (the counts 2, 4, 10, 24, 66 for 2–6 resistors are the classic series-parallel network sequence):

Reduction steps	Distinct shapes with 3–6 resistors	Largest one-difficulty bank
1	8	8
2	20	20
3	34	34
4	30	30
5	12	12

At exactly 3 steps there are 34 shapes with 4–6 resistors; restricting to 5–6 resistors (so the arithmetic load is also held in a narrow band) leaves exactly 30 — the bank uses every one. Wanting more items at one difficulty means allowing more resistors, not inventing more shapes — and that is a different difficulty. The first bank (10 items, mixed 2–5 steps) is kept as a contrast: structurally isomorphic, but not fair as a random draw.

SLIDE 7

7. Results and verification

Bank	Items	Difficulty	Proposer	Critic rejections	Answers
PHY2-DC-SPRTC-10222026	10	mixed, 2–5 steps	LLM	0 (10 distinct shapes)	several repeat
PHY2-DC-SPRTC3-10292026	30	exactly 3 steps, 5–6 resistors	enumerator	0 (all 30 shapes)	all distinct

The 30-item bank

id	circuit (lingo)	V	answer
q1	`12 || 4 || (10 || 10 + 1)`	20 V	10.00 A
q2	`12 || (4 || 4 + 1 + 1)`	36 V	12.00 A
q3	`3 || (8 || 4 || 8 + 4)`	18 V	9.00 A
q4	`6 || (6 + 6) || (8 + 4)`	6 V	2.00 A
q5	`(2 + 6) || (12 + 2 + 10)`	15 V	2.50 A
q6	`2 || 2 + 5 || 5 + 9`	6 V	0.48 A
q7	`12 || 12 + 12 || 4 || 6`	18 V	2.25 A
q8	`4 || 6 || (3 + 1) + 6`	30 V	4.00 A
q9	`12 || (8 + 4) + 10 + 9`	20 V	0.80 A
q10	`12 || (2 + 1 + 1) + 2`	15 V	3.00 A
q11	`4 || 12 || 12 || (3 || 6 + 10)`	36 V	18.00 A
q12	`3 || 4 || (2 || 2 + 5 + 6)`	30 V	20.00 A
q13	`4 || 12 || (2 || 3 || 6 + 2)`	9 V	6.00 A
q14	`8 || 8 || (9 + 3) || (1 + 2)`	12 V	8.00 A
q15	`9 || (4 || 4 + 2 + 12 + 2)`	30 V	5.00 A
q16	`5 || (3 || 2 || 6 + 10 + 9)`	18 V	4.50 A
q17	`12 || (8 || 8 || 4 || 2 + 5)`	30 V	7.50 A
q18	`4 || (5 + 5) || (10 + 4 + 6)`	36 V	14.40 A
q19	`(1 + 9) || (10 + 6 + 12 + 2)`	36 V	4.80 A
q20	`(3 + 8 + 9) || (12 + 2 + 6)`	9 V	0.90 A
q21	`9 || 9 + 3 || 3 + 8 + 1`	18 V	1.20 A
q22	`4 || 4 + 6 || 6 || 6 + 6`	6 V	0.60 A
q23	`12 || 6 + 12 || 12 || 3 || 2`	12 V	2.40 A
q24	`10 || 5 || 10 + 6 || 3 || 6`	15 V	3.75 A
q25	`6 || 6 || 9 || (9 + 9) + 10`	6 V	0.50 A
q26	`10 || 5 || (5 + 5) + 6 + 4`	36 V	2.88 A
q27	`10 || 12 || (4 + 5 + 6) + 4`	6 V	0.75 A

id	circuit (lingo)	V	answer
q28	`3 || (3 + 3) + 8 + 3 + 2`	6 V	0.40 A
q29	`10 || (5 + 3 + 2) + 12 + 8`	18 V	0.72 A
q30	`12 || (12 + 5 + 3 + 4) + 1`	9 V	1.00 A

Verification

Three independent checks were run. (1) Chen's own `validate_input.py` accepts the circuit input scheme with no missing fields. (2) Every accepted item was re-parsed and re-solved by the circuit engine, and the bank regenerated from its sidecar is byte-identical to the first build. (3) Each item's canonical shape is unique and has exactly three reduction steps — the critic's log shows 30 kept, 0 rejected, in a single pass. The banks are published in Chen's folder convention (`Subject-Unit-Abbreviation-date`, `status: ready`) in the `circuit-banks` repository.

SLIDE 8

8. Limits and next steps

Topic	Status
Canvas import (QTI)	Not produced. Chen's bank folders ship a QTI zip made by an external converter; his creation repo does not generate it. A deterministic QTI 2.1 exporter for multiple-choice-with-image is a well-scoped next step.
Varying the unknown	Fixed to “total current”. Chen's input has `unknown_rotation` for this but his pipeline does not implement it yet; our critic already treats a different ask as a different problem, so this is an input-file change plus wording.
Number randomisation per student	Not done (also “not yet” in Chen's docs). Isomorphism is carried by the topology; numbers are fixed per item.
Distractor magnitude	Left honest (no cap); one option can be 100× the answer. Documented as a design choice.
Licensing	Chen's repo is CC BY-NC 4.0 and is now private; the skill that embeds one of his scripts is kept private too. The public `circuit-banks` repo contains only our output.

Appendix A — all 3-step shapes with 5–6 resistors (the 30-item bank uses every one)

#	R	shape
1	5	`R || R || (R || R + R)`
2	5	`R || (R || R + R + R)`
3	5	`R || (R || R || R + R)`
4	5	`R || (R + R) || (R + R)`
5	5	`(R + R) || (R + R + R)`
6	5	`R || R + R || R + R`
7	5	`R || R + R || R || R`
8	5	`R || R || (R + R) + R`
9	5	`R || (R + R) + R + R`
10	5	`R || (R + R + R) + R`
11	6	`R || R || R || (R || R + R)`
12	6	`R || R || (R || R + R + R)`
13	6	`R || R || (R || R || R + R)`
14	6	`R || R || (R + R) || (R + R)`
15	6	`R || (R || R + R + R + R)`

#	R	shape
16	6	`R || (R || R || R + R + R)`
17	6	`R || (R || R || R || R + R)`
18	6	`R || (R + R) || (R + R + R)`
19	6	`(R + R) || (R + R + R + R)`
20	6	`(R + R + R) || (R + R + R)`
21	6	`R || R + R || R + R + R`
22	6	`R || R + R || R || R + R`
23	6	`R || R + R || R || R || R`
24	6	`R || R || R + R || R || R`
25	6	`R || R || R || (R + R) + R`
26	6	`R || R || (R + R) + R + R`
27	6	`R || R || (R + R + R) + R`
28	6	`R || (R + R) + R + R + R`
29	6	`R || (R + R + R) + R + R`
30	6	`R || (R + R + R + R) + R`

“R” = a resistor; | | parallel, + series; sibling order is irrelevant. Each row is a different reduction sequence of exactly three steps.

Appendix B — running it

In a chat with the skills installed: “run the circuit bank workflow on circuit-input.yaml”. Under the hood (repo root of the fork):

```
PY=.venv/bin/python # repo venv
P=.cursor/skills/problem-creation/scripts # Chen's scripts
S=.cursor/skills/circuit-bank/scripts # circuit-bank scripts
IN=BANK/circuit-input.yaml; SC=BANK/circuit-input.scenarios.yaml

$PY $P/validate_input.py $IN # Agent 1 check (Chen)
$PY $P/scenario_list.py init --input $IN # sidecar, target (Chen)
$PY $P/scenario_list.py receive --file $SC # start a pass (Chen)
$PY $S/enumerate_lingo.py --file $SC # propose: every shape in the band
$PY $S/add_lingo.py --file $SC --json BANK/agent2.json
$PY $S/critic.py --file $SC --apply # Agent 3 as a script
$PY $S/assemble_bank.py --input $IN --file $SC --out BANK # Agents 4+5 via circuit
skill
$PY $S/bank_to_pdf.py --bank BANK/<bank_id>.yaml
```

Files a bank folder contains (Chen's layout):

File	Role
`circuit-input.yaml`	the recipe: family, specs, count, ranges, difficulty, seed
`circuit-input.scenarios.yaml`	Chen's sidecar: accepted lingo (+ rejected with reasons) — the source of truth
`<bank_id>.yaml`	the bank in Chen's template: items, options, worked solutions — the importable product
`figures/qN.png`	one figure per item
`<bank_id>.pdf`	printable worksheet + answer key (for humans; students never see the whole bank)

Repositories: `myrlaphil/circuit-diagram-skill` (public, the engine) · `myrlaphil/circuit-bank-skill` (private, standalone glue) · fork of `Zhongzhou/isomorphic-bank-creation-skills`, branch `circuit-bank` (private, glue installed beside his skills) · `myrlaphil/circuit-banks` (public, banks only).